

Python编程基础



黑马程序员
www.itheima.com

传智教育旗下
高端IT教育品牌

Python 是什么?

Jun 2024	Jun 2023	Change	Programming Language		Ratings	Change
1	1			Python	15.39%	+2.93%
2	3	^		C++	10.03%	-1.33%
3	2	v		C	9.23%	-3.14%
4	4			Java	8.40%	-2.88%
5	5			C#	6.65%	-0.06%
6	7	^		JavaScript	3.32%	+0.51%
7	14	^^		Go	1.93%	+0.93%
8	9	^		SQL	1.75%	+0.28%
9	6	v		Visual Basic	1.66%	-1.67%
10	15	^^		Fortran	1.53%	+0.53%

Python 是一门快速增长的计算机编程语言

测试人员为什么要学习编程?



测试为什么要学习编程?

如果能看懂开发人员写的代码是不是可以测试的更好?



是的!

白盒测试



测试为什么要学习编程?

“点点点”的测试效率是不是比较低?



是的!

自动化测试可以提高效率，但是需要学习编程!



自动化测试



测试为什么要学习编程?

想不想为公司创造更高的价值?



做梦都想!

做测试开发，开发一些平台和工具让不懂编程的人使用



测试开发

为什么选择学习Python?



相对其他编程语言**更简单**

语言**开源**并且**免费**

适用人群**广泛**

应用领域**广泛**

学习目标

Learning Objectives

1. 能使用 Python 完成**自动化脚本**的**编写**和**管理**

```
def test_01_login(self):
    # 1. 输入用户名
    self.driver.find_element(By.ID, 'username').send_keys('15000000000')
    # 2. 输入手机号
    self.driver.find_element(By.ID, 'password').send_keys('123321')
    # 3. 输入验证码
    self.driver.find_element(By.ID, 'verify_code').send_keys('8888')
    # 4. 点击登录
    self.driver.find_element(By.NAME, 'sbtbutton').click()
    time.sleep(1)
    # 获取弹窗内容
    value = self.driver.find_element(By.CLASS_NAME, 'userinfo').text
    # 断言
    assert "15000000000" == value
```

```
def test_02_order(self):
    # 搜索 小米
    self.driver.find_element(By.CSS_SELECTOR, '#q').send_keys('小米')
    self.driver.find_element(By.CSS_SELECTOR, '.search_usercenter_btn').click()
    time.sleep(1)
    # 在结果页面 点击第一个商品 进入商品详情页
    self.driver.find_element(By.CSS_SELECTOR, '.xs_img').click()
    time.sleep(1)
    # 在商品详情页 点击 立即购买
    self.driver.find_element(By.CSS_SELECTOR, '#buy_now').click()
    time.sleep(1)
    # 处理弹窗
    self.driver.switch_to.alert.accept()
    time.sleep(1)
    # 点击提交订单
    self.driver.find_element(By.CSS_SELECTOR, '#submit_order').click()

    # 断言
    text = self.driver.find_element(By.CSS_SELECTOR, '.erhuh').text
    assert "订单提交成功, 请您尽快付款" in text
    assert '241.90' in text
```

Summary

10 tests ran in 38.19 seconds.

(Un)check the boxes to filter the results.

☒ 10 passed, ☐ 0 skipped, ☐ 0 failed, ☐ 0 errors, ☐ 0 expected failures, ☐ 0 unexpected passes

Results

[Show all details](#) / [Hide all details](#)

▲ Result	▼ Test	▼ Duration
Passed (show details)	script/test_tpshop_login.py::TestTpShopLogin::test_login[用户名为空--123321-8888-False-用户名不能为空]	6.44
Passed (show details)	script/test_tpshop_login.py::TestTpShopLogin::test_login[手机号未注册-15000000009-123321-8888-False-账号不存在]	1.65
Passed (show details)	script/test_tpshop_login.py::TestTpShopLogin::test_login[10位手机号-1500000000-123321-8888-False-账号格式不匹配]	1.64
Passed (show details)	script/test_tpshop_login.py::TestTpShopLogin::test_login[11位非数字-1500000000a-123321-8888-False-账号格式不匹配]	1.60
Passed (show details)	script/test_tpshop_login.py::TestTpShopLogin::test_login[密码为空-15000000000--8888-False-密码不能为空]	1.65
Passed (show details)	script/test_tpshop_login.py::TestTpShopLogin::test_login[密码错误-15000000000-123456-8888-False-密码错误]	1.64
Passed (show details)	script/test_tpshop_login.py::TestTpShopLogin::test_login[验证码错误-15000000000-123321-1234-False-验证码错误]	1.64
Passed (show details)	script/test_tpshop_login.py::TestTpShopLogin::test_login[登录成功-15000000000-123321-8888-True-15000000000]	2.89
Passed (show details)	script/test_tpshop_order.py::TestTpShopOrder::test_01_login	7.05
Passed (show details)	script/test_tpshop_order.py::TestTpShopOrder::test_02_order	11.70

测试报告



目录

Contents

◆ Python基础

◆ 数据序列

◆ 函数

◆ 异常

学习目标

Learning Objectives

1. 能够完成 Python 环境搭建
2. 能够正确定义变量及查看变量类型
3. 能够实现程序的输出和输入
4. 能够正确使用运算符
5. 熟练掌握流程控制语句：if、while、for



目录

Contents

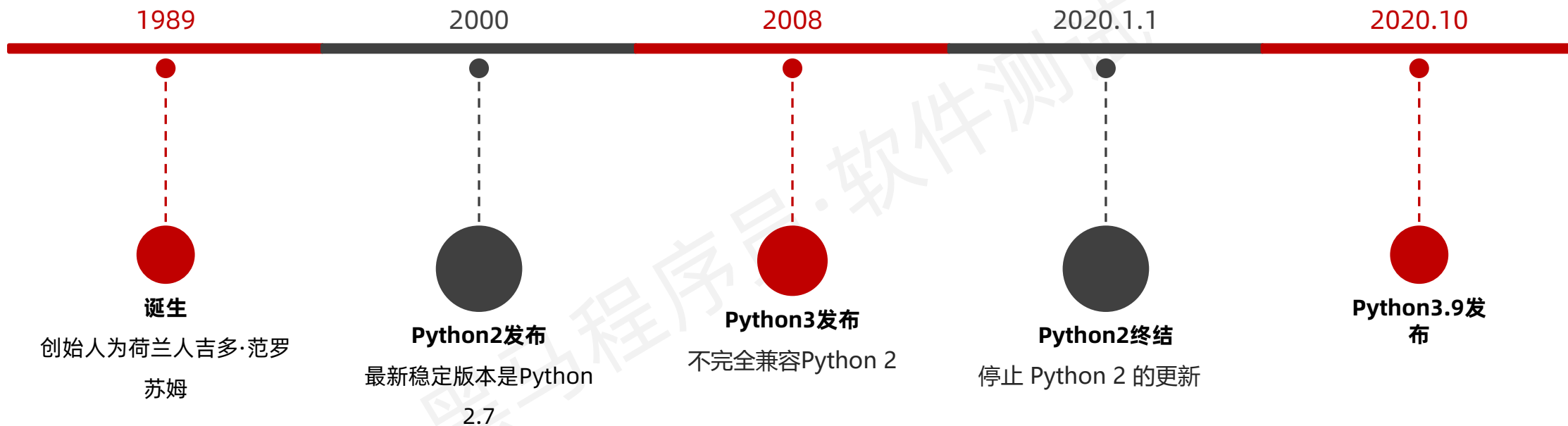
- ◆ 认识 Python
- ◆ Python 环境搭建
- ◆ 基础语法
- ◆ 程序的输出和输入
- ◆ 运算符
- ◆ 流程控制

01

认识 Python

- 内容
 - Python 发展史
 - 语言分类介绍
- 目标
 - 知道目前Python的主流版本
 - 知道Python语言的类型

认识 Python - Python 发展史



注意 本课程使用的版本：**Python 3.11.9**

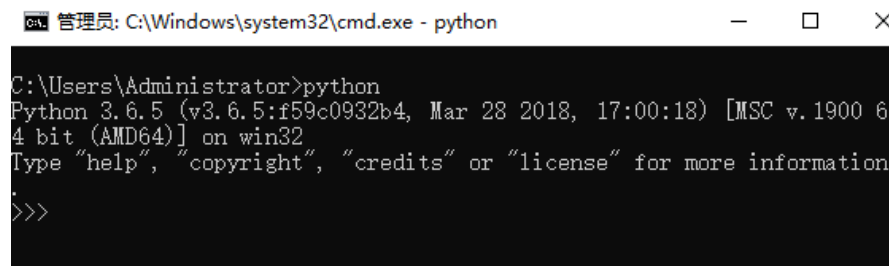
Python环境搭建

安装Python解释器

提示：Python 是一门解释型语言，通过解释器来运行 .py 文件

操作步骤：

1. 下载解释器安装包
 - 下载地址：<https://www.python.org/downloads/release>
2. 安装
 - 双击安装包开始安装
 - 选择自定义安装
 - 勾选添加环境变量
3. 验证
 - cmd进入命令行，输入python 查看信息



Python环境搭建

安装PyCharm

PyCharm介绍:

- Pycharm 是一种 Python IDE (集成开发环境), 内置功能丰富提高开发效率
- Pycharm 分为专业版(professional) 和 社区版(community)

提示: 课程中使用社区版

操作步骤:

1. 下载安装包
 - 下载地址:
<http://www.jetbrains.com/pycharm/download>
2. 安装
 - 双击安装包选择默认选项即可完成安装

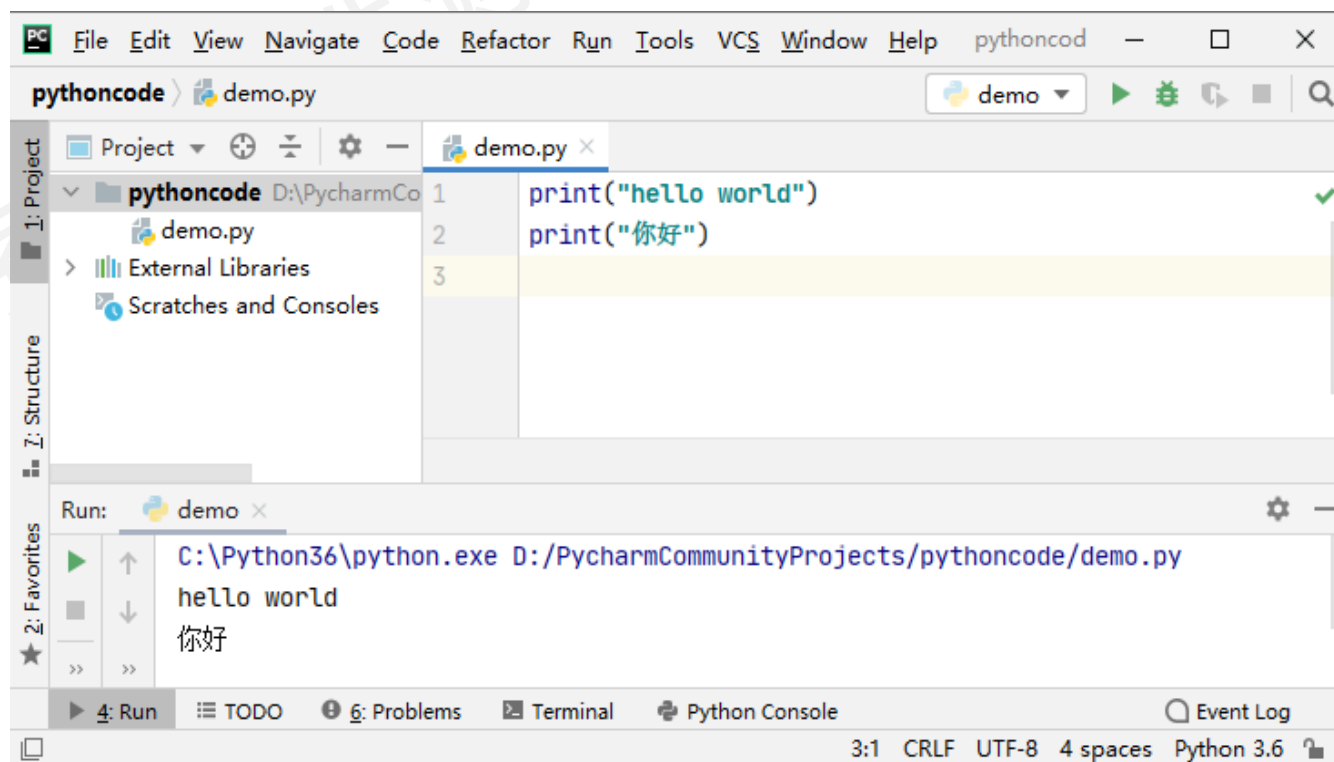


Python环境搭建

PyCharm中编写HelloWorld程序

操作步骤:

1. 启动PyCharm
2. 创建项目
3. 在项目目录下创建python文件
4. 编写python代码
5. 在当前文件编辑区直接点击鼠标右键，选择Run
6. 在下方控制台中即可看到打印的内容





目录

Contents

- ◆ 认识 Python
- ◆ Python环境搭建
- ◆ 基础语法
- ◆ 程序的输出和输入
- ◆ 运算符
- ◆ 流程控制

注释

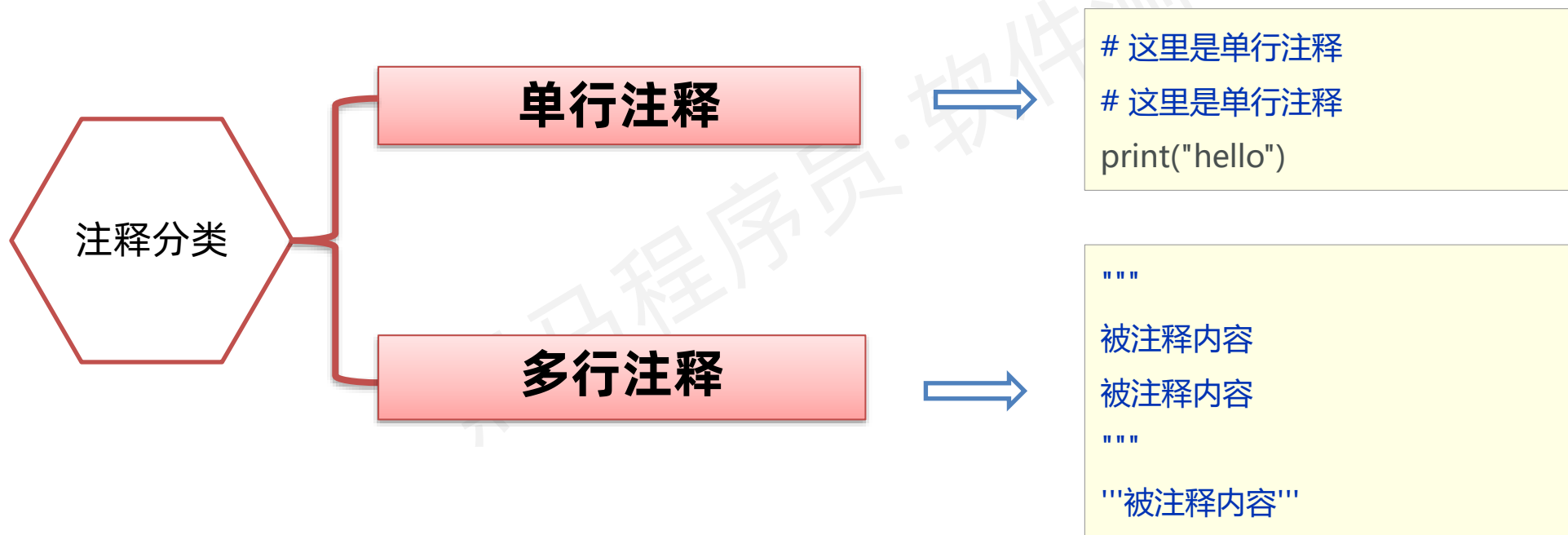
注释：就是对代码的解释和说明

目的：

- 让人们能够更加轻松地了解代码
- 增强代码的可读性

特点： **不会被解释器执行**

注释的分类



注释-扩展

注释快捷键: `ctrl + /`

添加注释的注意事项

- 注释并非越多越好，一目了然的代码无需注释
- 逻辑复杂的代码，应当先写注释再编码
- 不要用中文去翻译 python 代码的含义

基础语法

变量

概念 变量在计算机语言中能储存计算结果或表示某个数据值

变量的定义

变量名 = 数据值

说明:

- 变量名自定义, 要满足`标识符`命名规则
- 数据值为要存储的数据, 可以为不同的数据类型
- 单等号用于 赋值, 不具备比较功能

示例代码

```
name = "张三"  
age = 18  
print(name)  
print(age)
```

基础语法

标识符

概念 标识符是用户编程时使用的名字，用于给变量、函数、类等命名

标识符规则：

- 由数字, 字母, 下划线组成
- 不能使用数字开头
- 不能使用Python内置关键字
- 严格区分大小写
- 不建议使用中文

Python中的关键字

False None True and as assert break
class continue def del elif else except finally for
from global if import in
is lambda nonlocal not or pass raise return try
while with yield

基础语法

标识符-命名习惯

- **驼峰命名**

- 大驼峰：每个单词首字母大写, 例如: `MyName`
- 小驼峰：第二个单词开始首字母大写, 例如: `myName`

- **下划线命名**

- 每个单词之间使用下划线连接, 例如: `my\_name`



好的命名习惯要做到 **见名知意**

基础语法

数据类型

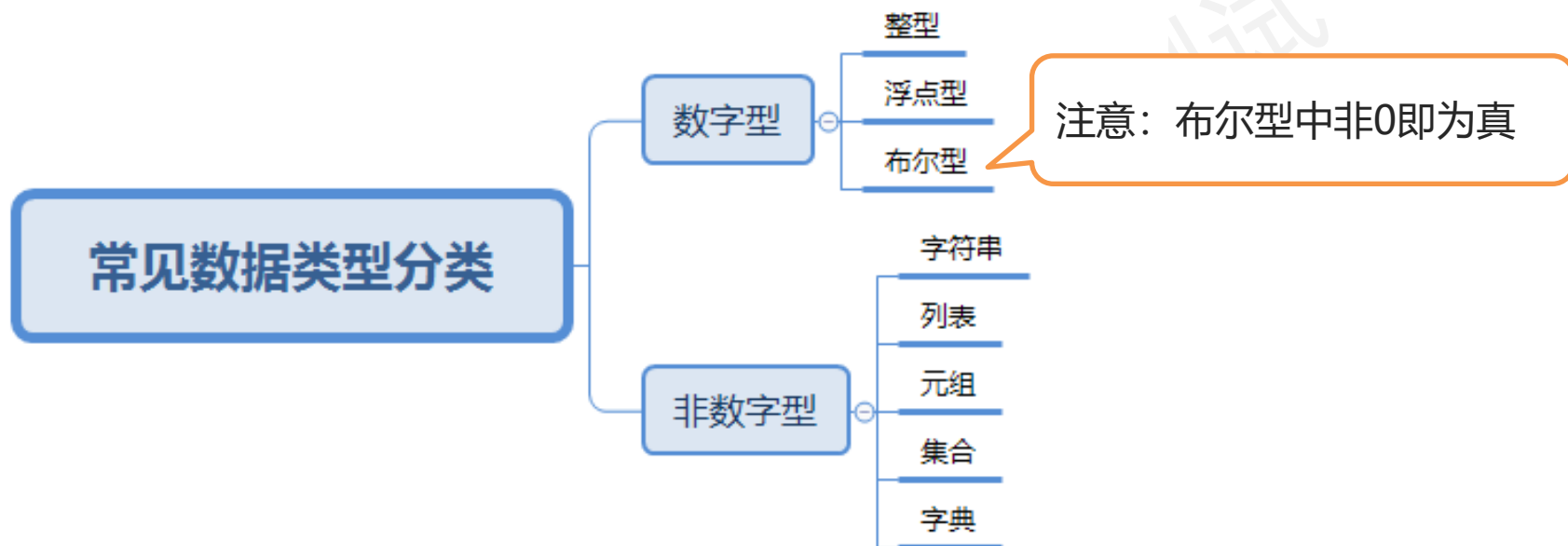


思考：数据为什么需要类型？

- 人类可以肉眼观察轻松的区分不同类型数据，但是计算机做不到
- 计算机工作的过程就是完成不同的类型的计算，例如做数学运算、做文件存储、做逻辑判断
- 数据类型可以对不同数据进行分类管理和标识

基础语法

数据类型分类



查看数据类型：

语法：**type(数据/变量名)**

注意：该函数本身不具备输出功能，需要配合print()函数使用！

```
print(type(18))
```

```
name = "张三"  
print(type(name))
```




总结

1. 注释的作用和分类?
2. 如何定义变量?
3. 标识符的命名规范有哪些?
4. 常见的数据类型有哪些?
5. 如何查看数据的类型?

程序的输出

程序的输出

语法: print(数据/变量)

说明 将程序的运行结果输出到控制台或终端窗口中

示例代码

```
print("张三")
```

```
age = 18
```

```
print(age)
```

程序的输出和输入

程序的输入

说明 在 Python 中接收用户输入的数据的过程为程序的输入

语法: **变量名 = input('提示信息:')**

注意事项

1. 当程序执行到 input() 函数会等待用户输入, 因此最好给出提示信息
2. input() 函数接收用户输入的信息后, 一般需要保存到到变量中, 方便后续使用
3. input() 函数会把用户输入的任意数据都当做字符串处理, 需要运算时注意类型转换

程序的输出和输入

格式化输出

说明 将程序的运行结果按照一定的格式输出

需求:

1. 已知两个变量
 - name = "小明"
 - age = 18
2. 在控制台输出 "小明今年18岁了"



```
name = "小明"  
age = 18  
  
print(name + "今年" + str(age) + "岁了")
```



存在的问题:

1. 字符串拼接，导致代码可读性差
2. 非字符串类型的数据，还要进行类型转换

程序的输出和输入

格式化输出-format方法实现

说明：调用字符串的format方法可以实现字符串的格式化

使用方式

```
name = "小明"  
age = 18  
  
print("name={} age={}".format(name, age))
```

使用说明

- {} 表示占位符，即要替换的位置
- format()方法中的参数为实际要替换的数据
- {} 的数量和参数的数量要一致

程序的输出和输入

格式化输出-f格式化实现

说明：f-格式化字符串是Python3.6中新增的格式化方法，该方法更简单易读

使用方式

```
name = "小明"
age = 18

print(f"name={name} age={age}")
```

使用说明

- 字符串前添加f标记
- {}中直接写变量名称

案例

程序输入案例

需求：

1. 提示用户输入两个数字
2. 使用获取到的数据进行加法运算
3. 在控制台输出计算结果，格式要求：XX + XX = XX

```
num1 = int(input("请输入第一个数字: "))  
num2 = int(input("请输入第二个数字: "))  
print(f"{num1} + {num2} = {num1 + num2}")
```

基础语法

数据类型转换

类型转换：将一种数据类型转换为另一种数据类型

函数	说明	注意
int(x)	将x转换为一个整数	x是字符串时，必须是 整数类型 的字符串
float(x)	将x转换为一个浮点数	x是字符串时，只要是 数字类型 的字符串即可
str(x)	将 x 转换为字符串	对于任意数据类型x都可以转为字符串类型
tuple(s)	将序列 s 转换为一个元组	
list(s)	将序列 s 转换为一个列表	

总结

1. 如何实现格式化输出?
2. 如何获取用户输入的数据信息?



目录

Contents

- ◆ 认识 Python
- ◆ Python环境搭建
- ◆ 基础语法
- ◆ 程序的输出和输入
- ◆ 运算符
- ◆ 流程控制

05

运算符

- 算术运算符
- 比较运算符
- 赋值运算符

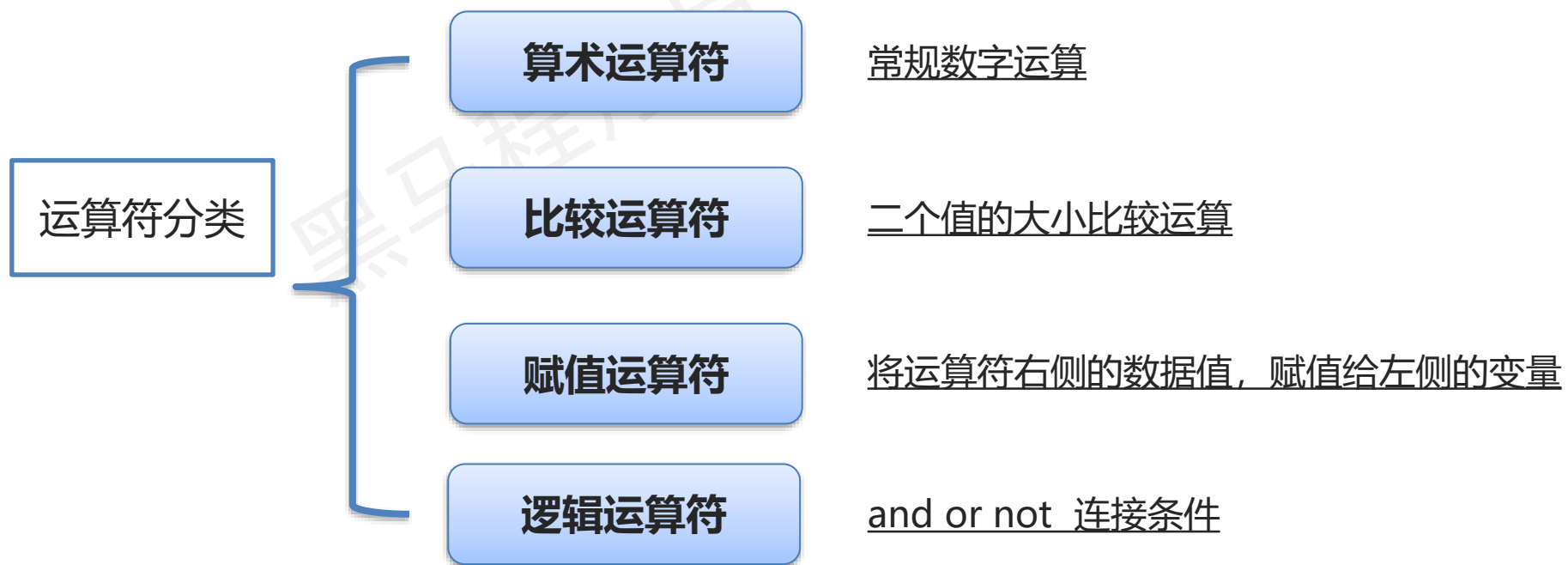
学习目标

1. 掌握如何使用运算符

运算符

运算符介绍

定义 运算符就是在各种运算中起到特定作用的符号



运算符

算术运算符

符号	作用	说明
+	加	参看小学一年级
-	减	参看小学一年级
*	乘	参看小学二年级，与“ $\times$ ”相同
/	除	参看小学二年级，与“ $\div$ ”相同
//	求商	获取的是两个数据做除法的商
%	取余	获取的是两个数据做除法的余数

运算符

比较运算符

符号	作用	说明
==	相等	检查运算操作数的值是否相等，如果成立返回True，否则返回False
!=	不相等	检查运算操作数的值是否不相等，如果成立返回True，否则返回False
>	大于	检查左侧操作数的值是否大于右侧，如果成立返回True，否则返回False
>=	大于等于	检查左侧操作数的值是否大于或等于右侧，如果成立返回True，否则返回False
<	小于	检查左侧操作数的值是否小于右侧，如果成立返回True，否则返回False
<=	小于等于	检查左侧操作数的值是否小于或等于右侧，如果成立返回True，否则返回False

注意事项

数字100 与 字符串100 是不相等的!!!

```
print(100 == "100") # False
```



注意：比较运算符的结果全部为**布尔值(True/False)**

案例

录入用户信息

需求：

1. 提示用户输入用户姓名，并保存到变量中
2. 提示用户输入用户年龄，保存到变量中，并转换成整数
3. 提示用户输入用户身高，保存到变量中，并转换成浮点数
4. 在控制台输出用户姓名、年龄、身高对应变量的数据类型
5. 按照以下格式输出用户信息：“姓名:xxx 年龄:xxx 身高:xxx”
6. 在控制台输出该用户5年之后的年龄，格式：“xxx 5 年之后的年龄是 xx”
7. 在控制台输出该用户现在是否成年，格式：“xxx是否成年: xxx”

总结

1. 运算符的分类有哪些?
2. 运算符的优先级?



目录

Contents

- ◆ 认识 Python
- ◆ Python环境搭建
- ◆ 基础语法
- ◆ 程序的输出和输入
- ◆ 运算符
- ◆ 流程控制

06

流程控制

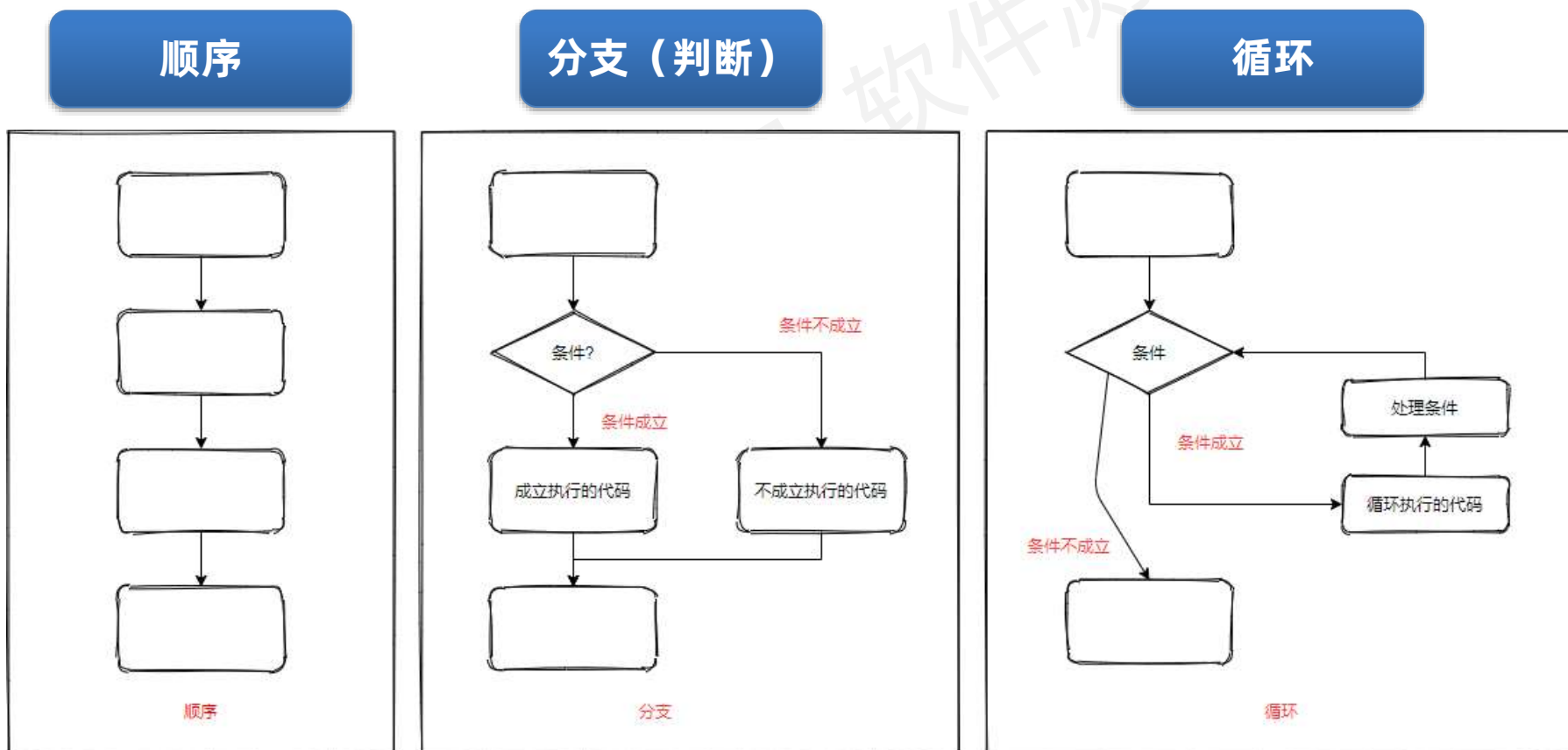
- 判断语句
- 循环语句

学习目标

1. 掌握if判断语句的用法
2. 掌握while循环的用法
3. 掌握for循环的用法

流程控制

程序执行的三大流程



流程控制-判断语句

判断语句的应用场景

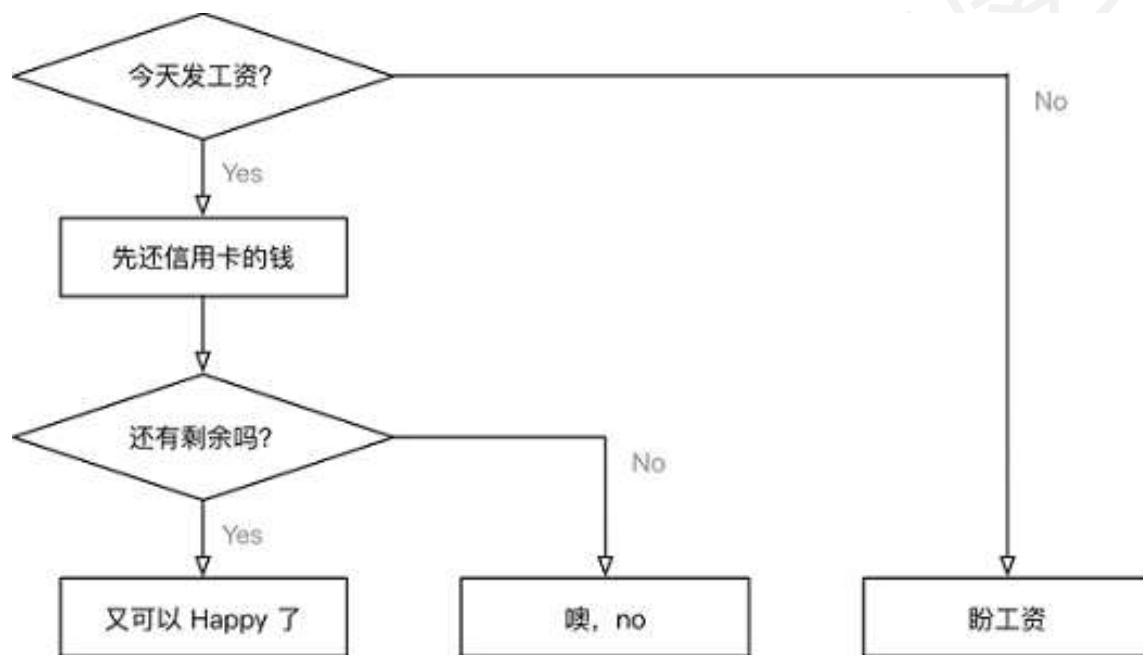


流程控制-判断语句

判断的定义：

- 如果条件满足，才能做某件事情，
- 如果条件不满足，就做另外一件事情，或者什么也不做

提示：判断语句涉及的关键字有 if、else、elif



```
if 今天发工资:
    先还信用卡的钱
    if 有剩余:
        又可以happy了~
    else:
        噢, no。。。还得等30天!
else:
    盼着发工资
```

流程控制-判断语句

if 判断语句基本语法

if 要判断的条件:

条件成立时, 要做的事情

.....



注意:

- 可以把整个 if 语句看成一个完整的代码块
- 代码的缩进为一个 tab 键, 或者 4 个空格
- 在 Python 开发中, Tab 和空格不要混用!

提示: PyCharm 中可以直接使用 Tab 键完成代码缩进, 因为默认设置会自动转化为 4 个空格

流程控制-判断语句

else 处理条件不满足的情况

if 要判断的条件:

条件成立时, 要做的事情

.....

else:

条件不成立时, 要做的事情

.....



注意:

1. if 和 else 语句以及各自的缩进部分共同是一个完整的代码块
2. else 关键字必须和 if 联合使用, 不能单独使用

案例

if-else判断语句-判断年龄

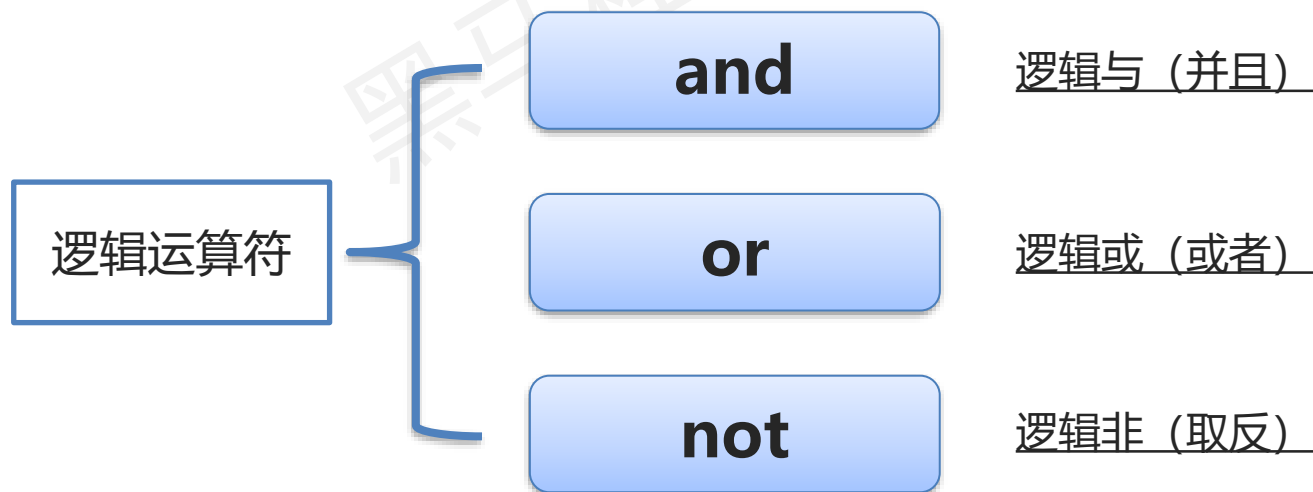
需求:

1. 输入用户年龄
2. 判断是否满 18 岁 ($\geq$)
3. 如果满 18 岁，允许进网吧嗨皮
4. 如果未满 18 岁，提示回家写作业

流程控制-判断语句

逻辑运算符

- 在程序开发中，通常在判断条件时，会需要同时判断多个条件
- 只有多个条件都满足，才能够执行后续代码，这个时候需要使用到 逻辑运算符
- 逻辑运算符 可以把 多个条件 按照 逻辑 进行 连接，变成 更复杂的条件



逻辑运算符-and

条件1 and 条件2

- 逻辑与 / 并且
- 两个条件同时满足，返回 True
- 只要有一个不满足，就返回 False

运算符

逻辑运算符-or

条件1 or 条件2

- 逻辑或 / 或者
- 两个条件只要有一个满足，返回 True
- 两个条件都不满足，返回 False

运算符

逻辑运算符-not

not 条件

- 逻辑非 / 取反
- 如果True，not之后为 False
- 如果False，not之后为 True

运算符

运算符优先级

- 不同的运算符默认具备不同的优先级
- 想把所有的运算符优先级都记住确实没必要



请记住：**小括号的优先级最高**，当无法确定谁的优先级高时，加一个小括号就解决了

案例

逻辑运算符-and

需求:

1. 获取用户输入的用户名和密码
2. 判断用户名是 admin 并且密码是 123456 时, 在控制台输出: 登录成功!
3. 否则在控制台输出: 登录信息错误!

案例

逻辑运算符-or

需求:

1. 定义两个整数变量python_score、c_score，编写代码判断成绩
2. 要求只要有一门成绩 > 60 分就算合格

 案例

判断闰年

需求:

1. 闰年判断程序(闰年是能被4整除，但不能被100整除的；或者能被400整除的年份)
2. 输入一个有效的年份，判断是不是闰年 year
3. 如果是闰年，则打印“xxxx年是闰年”；否则打印“xxxx年不是闰年”
4. 如输入"2018"，将打印“2018年不是闰年”

流程控制-判断语句

elif 多重判断

```
if 条件1:  
    条件1满足执行的代码  
    .....  
elif 条件2:  
    条件2满足时, 执行的代码  
    .....  
elif 条件3:  
    条件3满足时, 执行的代码  
    .....  
else:  
    以上条件都不满足时, 执行的代码  
    .....
```



应用场景：在判断条件时, 需要判断多个条件, 并且对应不同条件要执行不同的代码

提示：elif 和 else 都必须和 if 联合使用，而不能单独使用

案例 elif 多重判断

需求:

1. 定义 score 变量记录考试分数
2. 如果分数是 大于等于 90分 显示 优
3. 如果分数是 大于等于 80分 并且 小于 90分 显示 良
4. 如果分数是 大于等于 70分 并且 小于 80分 显示 中
5. 如果分数是 大于等于 60分 并且 小于 70分 显示 差
6. 其它分数显示 不及格

```
score = 49

if score >= 90:
    print("优")
elif score >= 80 and score < 90:
    print("良")
elif score >= 70 and score < 80:
    print("中")
elif score >= 60 and score < 70:
    print("差")
else:
    print("不及格")
```

流程控制-判断语句

if的嵌套

if 条件1:

 条件1 满足执行的代码

 if 条件1基础上的条件2:

 条件2 满足时，执行的代码

 # 条件2不满足的处理

 else:

 条件2 不满足时，执行的代码

条件1 不满足的处理

else:

 条件1 不满足时，执行的代码



应用场景：在之前条件满足的前提下，再增加额外的判断



案例 if的嵌套

需求:

1. 定义布尔型变量 has_ticket 表示是否有车票
2. 定义整型变量 knife_length 表示刀的长度，单位：厘米
3. 首先检查是否有车票，如果有，才允许进行安检
4. 安检时，需要检查刀的长度，判断是否超过 20 厘米
 - 如果超过 20 厘米，提示刀的长度，不允许上车
 - 如果不超过 20 厘米，安检通过
5. 如果没有车票，不允许进门

```
has_ticket = True # 表示是否有车票
knife_length = 20 # 刀的长度

# 首先检查是否有车票，如果有，才允许进行 安检
if has_ticket:
    print("有车票，可以开始安检...")
    # 如果超过 20 厘米，提示刀的长度，不允许上车
    if knife_length >= 20:
        print(f"不允许携带 {knife_length} 厘米长的刀上车")
    # 如果不超过 20 厘米，安检通过
    else:
        print("安检通过，祝您旅途愉快.....")
# 如果没有车票，不允许进门
else:
    print("大哥，您要先买票啊")
```

流程控制-判断语句



思考

1. 判断语句的作用是?
2. 判断语句的语法形式有哪些?

流程控制-循环语句

循环介绍

循环的作用 就是让 **指定的代码** 重复的执行

需求: 假如我有个女朋友，有一天我们闹矛盾生气了，女朋友说：道歉，说100遍“媳妇儿，我错了”。这个时候程序员会怎么做？



敲100遍 `print('媳妇儿，我错了')`



敲1遍，复制粘贴100次



重复执行100次一样的代码，程序中循环即可

流程控制-循环语句

while语句基本语法

#设置计数器(通常定义一个整数变量, 用来记录循环开始的数字)

i = 1

#while 循环条件: (判断计数器是否达到循环次数)

while i <= 5: #5, 设置循环5次

 条件满足时, 做的事情1

 条件满足时, 做的事情2

 条件满足时, 做的事情3

 ...(重复代码)...

计数器 + 1

 i = i + 1

print("终于循环完了")

条件成立, 执行分支代码; 然后再判断条件, 形成循环

条件不成立, 跳出循环, 执行后续代码



应用场景: while 循环最常用的应用场景就是 让执行的代码 按照 指定的次数 重复执行

第一次循环: 循环之前i=1
 循环之后i=2
第二次循环: 循环之前i=2
 循环之后i=3
第五次循环: 循环之前i=5
 循环之后i=6
第六次循环: 循环之前i=6

目的: 当达到循环次数之后, 跳出循环, 执行之后的代码
实现方式: 每一次循环完成之后, 将计数器+1

语法格式:

#1.设置计数器

#2.设置while条件

#3.重复执行的代码

#4.计数器+1

 案例

while循环

需求: 计划吃10个苹果，打印"吃第1个苹果","吃第2个苹果",..., "吃第10个苹果"

分析1: 是否需要重复执行的代码?

`print( "吃第X个苹果" )`

X等同于第几次循环

分析2: 需要循环几次?

10次

总结: 需要将指定代码重复执行多次，可以使用while循环语句来完成

运算符

赋值运算符

符号	作用	说明
=	赋值	a=10, 将10赋值给变量a
+=	加后赋值	a+=b, 将a+b的值给a, 相当于a=a+b
-=	减后赋值	a-=b, 将a-b的值给a
=	乘后赋值	a=b, 将a×b的值给a
/=	除后赋值	a/=b, 将a÷b的商给a
%=	取余后赋值	a%=b, 将a÷b的余数给a

注意事项

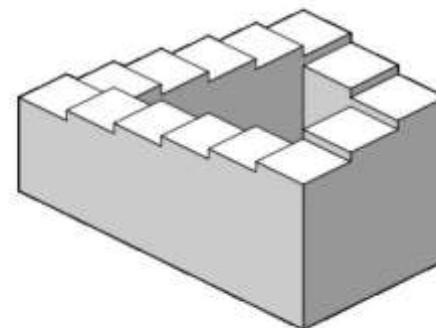
1. = , 单等号在程序语言中不表示比较, 用于赋值
2. 复合赋值运算符之间没有空格, 连接在一起

流程控制-循环语句

死循环

```
i = 1  
  
while i <= 10:  
    print("吃第{}个苹果".format(i))  
  
print("该代码会执行到吗? ")
```

根本停不下来



由于循环结束条件始终满足, 程序持续执行起来不会停止的现象, 称为**死循环**

提示 死循环, 通常是一个BUG

流程控制-循环语句

break 关键字

注意：break 是专门在循环语句中使用的关键字

break: 当某条件满足时, 退出当前循环语句

需求:

1. 一共吃5个苹果，吃完第一个，吃第二个...，这里"吃苹果"的动作是不是重复执行？
2. 如果吃的过程中，吃完第三个吃饱了，则不需要再吃第4个和第五个苹果，即是吃苹果的动作停止

```
i = 1
```

```
while i <= 5:
```

```
    print("吃了第%d个苹果" % i)
```

```
    if i == 3:
```

```
        print("吃饱了不吃了")
```

```
        break
```

```
    i += 1
```

流程控制-循环语句

For 指定次数的循环

说明: for 循环想要实现循环特定次数, 可以配合 rang() 方法实现

for 临时变量 in **range(循环次数)**:
 循环内部执行的代码

注意: 循环次数为整数类型数据

流程控制-循环语句

for 遍历

```
for 临时变量 in 容器:  
    重复执行的代码1  
    重复执行的代码2  
    ...
```

说明:

- for循环也可以让指定的代码重复执行
- for循环可以遍历容器中的数据
- 遍历: 从容器中把数据一个一个取出
- 容器: 字符串/列表/元组/字典数据

流程控制



思考

1. if判断语句的用法?
2. while循环的用法?
3. for循环的用法?

Python基础



总结

1. 能够完成 Python 环境搭建
2. 能够正确定义变量及查看变量类型
3. 能够实现程序的输出和输入
4. 能够正确使用运算符
5. 熟练掌握流程控制语句：if、while、for



目录

Contents

◆ Python基础

◆ 数据序列

◆ 函数

◆ 异常

学习目标

Learning Objectives

1. 掌握字符串的常用操作
2. 掌握列表的作用和操作方法
3. 掌握元组的作用和操作方法
4. 掌握字典的作用和操作方法
5. 掌握函数的用法

01

字符串

字符串

定义：使用引号(单引号,双引号,三引号)引起来的内容就是字符串。

引用：通过索引（下标）获取字符串数据。

索引(下标)：是数据在容器中的位置编号。对于字符串来说,就是字符在整个字符串中的位置。

0	1	2	3	4	5	6	正数索引
a	b	c	d	e	f	g	字符串
-7	-6	-5	-4	-3	-2	-1	负数索引

说明

- 1、一般情况下,使用的都是正数索引(从左到右), **从 0 开始的**。
- 2、Python 中支持负数索引(从右到左), **-1 表示容器中最后一个数据**。
- 3、使用索引(下标) 可以获取容器中指定位置的数据, 如: **字符串[索引]**。
- 4、获取字符串长度: **len(字符串)**

 案例

字符串基础操作

需求:

1. `str1 = 'abcdefg'`
2. 打印字符串中第一个位置的字符
3. 打印 字符串中最后一个位置的字符
4. 打印字符串长度
5. 分别使用 `for` 循环 和 `while` 循环对字符串进行遍历

字符串

查找 find()

字符串.find(被查找字符)

说明

被查找字符是否存在于当前字符串中, 如果存在则返回开始下标, 不存在则返回 -1

 案例

字符串查找

需求:

1. 现有字符串数据: 'itcast@qq.com'
2. 请设计程序, 实现判断 "@" 符号是否存在于数据中
3. 要求如果数据存在, 则输出数据所在位置否则打印输出"邮箱格式错误"

```
email = input("请输入你的邮箱: ")  
if email.find("@") != -1:  
    print(email.find("@"))  
else:  
    print("邮箱格式错误")
```

字符串切片

字符串[开始位置下标:结束位置下标:步长]

说明

- 1、通过切片操作，可以获取字符串中指定部分的字符。
- 2、结束下标位置对应的字符不会被截取到
- 3、步长用于设置截取间隔，默认步长为1

 案例

字符串切片操作

需求:

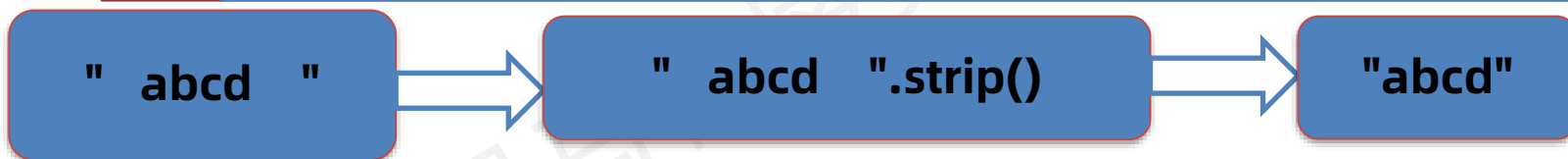
1. 定义字符串str1 = 'abcdefg'
2. 使用切片打印输出'cde'
3. 使用切片打印输出'abcde'
4. 使用切片打印输出'bcdefg'
5. 使用切片打印输出'abcdefg'
6. 使用切片打印输出'aceg'
7. 使用切片打印输出'abcdef'

字符串

去除两端空白字符 strip

字符串.strip()

说明 将字符串两端的空白字符(空格、换行\n、制表符\t)去除，返回一个新的字符串



问题：

登录成功之后，校验返回的信息为“恭喜你，登录成功”

实际操作完成之后，获取信息为 “ 恭喜你，登录成功 ”

这是一个BUG嘛？

如何使用代码来自动进行校验？

 案例

字符串去除空白字符

需求:

1. 现有字符串数据: ' 15000000000 '
2. 请设计程序, 去除字符串两端的空白字符

```
username = ' 15000000000 '  
username = username.strip()  
print(username)
```

字符串

字符串转换大小写 lower、upper

字符串.lower()

字符串.upper()

说明 将字符串转换为大写或者小写， 返回新的字符串

字符串

拆分 split()

字符串.split(分割符)

说明 按照指定字符来分割字符串

注意:

1. 方法执行完成后返回的数据类型为列表(list)
2. 不传入分割符时，默认以空白字符进行拆分

 案例

字符串大小写转换

需求：

1. 数据常见操作：增（insert）删（delete）改（update）查（select）

测试数据：

- Insert into student values(001, "jack", 18);
- delete from student where id=001;
- UPDATE student set name='rose'where id=001;
- Select * from student where id=001;

2. 请设计程序实现：

- a. 用户输入字符串为查询操作时，打印“显示查询结果”；
- b. 否则，提醒“当前进行的是是非查询操作，需要进行事务提交与回滚”。

字符串的其他常见方法

替换：字符串.replace(原字符串, 新子字符串)

说明 使用新的子字符串, 按规则替换旧的字符串内容

连接：字符串.join(一般为列表)

说明 一般用于将列表按指定子字符合并为字符串

判断是否为纯数字：字符串.isdigit()

说明 返回值为 True 和 False



练习一

需求：

1. 现有字符串数据: 'hello Python and itcast and itheima'
2. 请设计程序:
 - ① 将and替换为or
 - ② 显示: hello+Python+and+itcast+and+itheima

练习二

需求：

1. 判断输入的数据是否为数字
2. 如果是数字，打印“你太棒了”
3. 如果不是数字，打印“你到底能不能行”

案例 登录

需求:

1. 用户输入手机号、密码、验证码
2. 对于输入的手机号、密码、验证码去除两端空格
3. 手机号为空时，提示“手机号不能为空”
4. 密码为空时，提示“密码不能为空”
5. 验证码为空时，提示“验证码不能为空”
6. 手机号格式不正确时，提示"手机号格式错误，请输入11位数字"
7. 验证码统一转换为小写进行判断，正确验证码为"8888"，验证码错误、提示"验证码与图片内容不一致"
8. 正确手机号为"13488888888"，手机号错误时，提示"手机号不存在"
9. 正确密码为"123456"，密码错误时，提示"密码与账号不匹配"
10. 手机号为"13488888888"、密码为"123456"、验证码为"8888"时，输出“登录成功”

账户登录

 手机号/邮箱

 密码

验证码 

[忘记密码?](#)

登 录



思考

目前的登录只能支持 一个账号的判断，如果有多个账号
需要进行判断怎么处理?

02

列表

- 列表的定义和基本操作
- 列表的增删改查操作

列表

列表的定义

说明 列表(list)是 Python 中使用最频繁的数据类型, 在其他语言中通常叫做数组, 专门用来存储一组数据

定义方式

```
name_list = []  
name_list = ["tom", "jack", "lily"]
```

用[]定义, 数据之间使用英文逗号分隔

```
data_list = list()
```

通过类实例化方式定义

列表

列表增加方法：**append()**

注意：

- 1.方法执行是对原数据进行的修改，固列表是可变数据类型
- 2.如果增加一个列表，则此列表会被当做一个值添加到末尾

列表.append(新增数据)

说明 在列表的末尾添加数据

```
val_list = ["Web自动化", "UI自动化", "接口自动化"]
```

```
val_list.append("APP自动化")
```

```
print(val_list) # ['Web自动化', 'UI自动化', '接口自动化', 'APP自动化']
```

列表

列表删除方法：pop()

注意：使用方法时如果不传入索引值，默认删除列表中最后一个数据

del_data = 列表.pop(索引)

说明 删除指定索引对应的数据

```
val_list = ["Web自动化", "UI自动化", "接口自动化"]
```

```
val = val_list.pop(0)
```

```
print(val, val_list) # web自动化, ['UI自动化', '接口自动化']
```

列表修改方法：索引

注意：使用不存在的索引, 代码执行会报错!

列表[索引] = 新数据

说明 通过指定索引修改对应数据

```
val_list = ["Web自动化", "UI自动化", "接口自动化", "Web自动化"]  
  
val_list[1] = "黑马程序员"  
  
print(val_list) # ['Web自动化', '黑马程序员', '接口自动化', 'Web自动化']
```

列表

列表查询方法：索引

注意：使用不存在的索引，代码执行会报错！

item = 列表[索引]

说明 索引就是数据在列表中的位置编号，索引又被称为下标，默认从0开始

```
name_list = ['张三', '李四']  
print(name_list[0]) # 张三  
print(name_list[1]) # 李四
```

案例

登录案例优化

需求:

1. 定义**列表**存储测试账号
2. 遍历列表数据进行测试账号校验
3. 当手机号为"13488888888"，输出“账号正确”；否则“账号错误”。

账户登录

	手机号/邮箱
	密码
验证码	

[忘记密码?](#)

登 录

列表

列表反转方法：reverse()

注意：方法执行是对原数据进行的修改

列表.reverse()

说明 反转列表, 将列表中的元素倒序

```
num_list = [1, 2, 3, 4]
num_list.reverse()
print(num_list) # [4, 3, 2, 1]
```


列表

列表排序方法：sort()

注意：reverse 表示排序规则，默认是False表示升序，设置为True表示降序

列表.sort(reverse=False)

说明 将列表按指定规则进行数据排序，默认为升序

```
val_list = [8, 100, 30, 10, 40, 2]
```

```
val_list.sort(reverse=True)
```

```
print(val_list) # [100, 40, 30, 10, 8, 2]
```

列表

列表统计方法：**count()**

注意：如果目标数据不存在则返回 0

num = 列表.count(目标数据)

说明 统计被测试值出现的次数

```
data_list = ['python', 'java', 'python', 'php']
```

```
print(data_list.count("python")) # 2
```



思考

该如何存储更多的测试数据？ 比如：手机号、密码、验证码等

03

字典

- 字典的定义和基本使用
- 字典的增删改查
- 字典的遍历

字典

字典的定义

提示：大括号内结构为：
键名：键值【俗称键值对】

定义方式

```
data = {}  
data = {"name": "admin", "pwd":  
"123"}
```

用{}定义，数据之间使用
英文逗号分隔

```
data = dict()
```

通过类实例化方式定义

注意 字典中的键一般都使用字符串，并且键名不能重复（如果重复原数据会被覆盖）

字典

字典增加

提示：如果键不存在, 则增加键值对; 如果键存在, 则修改键对应的值

字典['键'] = 值

说明 在字典中增加键值对或修改已有键对应的值

```
info = {  
    "name": "tom",  
    "age": 18  
}  
info["salary"] = 100000  
  
print(info) # {'name': 'tom', 'age': 18, 'salary': 100000}
```

字典删除方法

字典.pop(键)

提示：如果给出的键在字典中不存在, 代码执行会报错!

说明 在字典数据中根据给出的键删除对应值

```
info = {  
    "name": "tom",  
    "age": 18,  
    "gender": "男"  
}  
info.pop("gender")  
print(info) # {'name': 'tom', 'age': 18}
```

字典

字典修改方法

提示：如果键不存在, 则增加键值对; 如果键存在, 则修改键对应的值

字典['键'] = 值

说明 在字典中增加键值对或修改已有键对应的值

```
info = {  
    "name": "tom",  
    "age": 18  
}  
info["salary"] = 100000  
  
print(info) # {'name': 'tom', 'age': 18, 'salary': 100000}
```


字典

字典查询方法：get()

value = 字典.get(键)

注意：如果查询的键在字典中不存在，代码执行会返回 None(空值)!

说明 通过键名查询对应值

```
info = {  
    "name": "tom",  
    "age": 18,  
    "gender": "男"  
}  
print(info.get("name")) # tom  
print(info.get("abc")) # None
```

字典

遍历字典的Key

```
for key in 字典.keys():
```

说明 循环拿到字典中的每个键名

```
info = {  
    "name": "tom",  
    "age": 18,  
    "gender": "男"  
}  
for key in info.keys():  
    print(key)
```

字典

遍历字典的Value

for value in 字典.values():

说明 循环拿到每个键对应的值

```
info = {  
    "name": "tom",  
    "age": 18,  
    "gender": "男"  
}  
for value in info.values():  
    print(value)
```

字典

遍历字典的Key和Value

for k, v in 字典.items():

提示：调用字典.items()方法获取字典的键和值，并自动赋值给不同的变量

说明 循环拿到每个键和值

```
info = {  
    "name": "tom",  
    "age": 18,  
    "gender": "男"  
}  
for k, v in info.items():  
    print(f"key={k} value={v}")
```

案例

登录案例优化

需求：

1、定义字典存储测试数据，包含账号和密码

提示：`{ "username" : "13488888888" , "password" : "123456" }` ,
`{ "username" : "" , "password" : "123456" }`

2、遍历数据进行测试账号和密码校验

3、当手机号为"13488888888"且密码为"123456"时，输出"登录成功"；否则"错误或密码错误"。

账户登录

	手机号/邮箱
	密码
验证码	

[忘记密码?](#)

登 录

04

元组

- 元组的定义
- 元组常用方法

学习目标

1. 掌握如何定义元组
2. 掌握元组的常用操作方法

元组的定义

说明 元组和列表一样，都可用于存储多个数据，不同之处在于**元组的元素不能修改**

定义方式

```
user_info = ()  
user_info = ("zhangsan", 18, 1.75)
```

用()定义，数据之间使用英文逗号分隔

```
info_tuple = tuple()
```

通过类实例化方式定义

注意：元组中只包含一个元素时，需要在元素后面添加逗号
data = (1,)

元组

元组查询方法：索引

注意：使用不存在的索引，代码执行会报错！

item = 元组[索引]

```
tuple1 = (1, 2, 3)
print(tuple1[1]) # 2
```


元组

元组查询方法: `count()`

注意: 如果目标数据不存在则返回 0

`num = 元组.count(目标数据)`

说明 统计被测试值出现的次数

```
tuple1 = (1, 2, 3)
print(tuple1.count(3)) # 1
```

元组的特殊用法：交换两个变量的值【面试题】

说明：

- 在 Python 中可以使用对应数据个数的变量, 获取对应元组数据中的每一个元素
- 在 Python 中定义多个元素的元组数据时, 小括号可以省略
- 借助以上两个特性, 可以通过元组快速实现交换两个变量的值

```
num1 = 100
num2 = 200
# 交换两个变量的值
num2, num1 = num1, num2
print(num1) # 200
print(num2) # 100
```

案例

登录案例优化

需求:

1. 定义元组存储测试账号
2. 遍历元组数据进行测试账号校验
3. 当手机号为"13488888888"，输出“账号成功”；否则“账号错误”。

账户登录

	手机号/邮箱
	密码
验证码	

[忘记密码?](#)

登 录

05

集合

集合的定义

说明 集合中的数据是没有重复的, 主要应用列表中的数据去重

定义方式

Set1 = {1, 2, 3}

用()定义, 数据之间使用
英文逗号分隔

set2 = set()

通过类实例化方式定义

案例

列表去重

1. 使用随机数生成 10 个 1-20 之间的随机数
2. 对列表中的数据进行去重

```
# 导包
import random

# 实现
test_data = []
for i in range(11):
    item = random.randint(1, 20)
    test_data.append(item)
print(test_data)
print(set(test_data))
```



目录

Contents

- ◆ Python基础
- ◆ 数据序列
- ◆ 函数
- ◆ 异常

06

函数

- 函数基本使用
- 函数参数
- 函数返回值

学习目标

1. 掌握如何定义函数
2. 掌握如何调用函数

函数

函数介绍

函数：把具有独立功能的代码块组织为一个小模块，在需要的时候调用



作用 对具备相同逻辑的代码进行封装, 提高代码的编写效率, 实现对代码的重用

函数

定义函数

语法格式

```
def 函数名(参数1, 参数2...):
```

函数封装的代码，可以使用传递进来的参数

```
pass
```

说明:

1. 在函数名的后面的小括号内部填写**参数**
2. 多个参数之间使用**,**分隔
3. 参数不是必须的，是可选的

说明:

1. def 是英文 define 的缩写
2. 函数名最好能够表达函数内部封装的代码的功能, 方便后续的获取调用[见名知意]
3. 函数名命名遵循标识符命名规则: 字母, 数字, 下划线, 不能以数字开头, 不使用系统关键字
4. 定义时候的参数称为形式参数, 简称**形参**

多一句没有，少一句不行，用更短时间，教会更实用的技术！

语法格式

函数名(参数1, 参数2...):

说明:

1. 在函数名的后面的小括号内部填写**参数**
2. 多个参数之间使用 **,** 分隔
3. 根据定义时是否有形参填写

说明:

1. 只定义函数, 不调用函数, 函数永远不会被执行
2. 不能将 函数调用 放在 函数定义 的上方, 否则将会出现错误
3. 函数调用时的参数称为实际参数, 简称**实参**

案例

函数实现登录案例

需求：当手机号为“13488888888”，输出“账号成功”；否则“账号错误”。

账户登录



手机号/邮箱



密码

验证码



[忘记密码?](#)

登 录

```
def login(phone):  
    if phone == "13488888888":  
        print("账号正确")  
    else:  
        print("账号错误")  
  
if __name__ == '__main__':  
    login("13488888888")  
    login("")
```

 练习

• 登录案例优化

需求：

1、定义**字典**存储测试数据，包含账号和密码

提示：`[{"username": "13488888888", "password": "123456"}, {"username": "", "password": "123456"}]`

2、遍历数据进行测试账号和密码校验

3、当手机号为“13488888888”且密码为“123456”时，输出“登录成功”；否则“错误或密码错误”。

账户登录

 手机号/邮箱

 密码

验证码



[忘记密码?](#)

登 录

思考

客户端怎么知道服务器中 login 函数的执行结果呢?

web 页面

账户登录

15000000000

.....

8888

忘记密码?

登 录

服务器

```
# 定义登录函数
def login(username, password, code):
    """
    """
    if code == '8888':
        if username == '15000000000' and password == '123456':
            print('登录成功')
        else:
            print('用户名或密码错误')
    else:
        print('验证码错误')
```

函数

函数的返回值

语法格式

```
def 函数名(参数...):  
    ...  
    return xxx
```

说明 如果想在函数调用处获取函数定义内部的某个数据, 需要使用 return 关键字返回结果

```
def demo():  
    name = "admin"  
    print(name) # admin  
  
result = demo()  
print(result) # None
```

错误样例: 没有返回值的函数,
在函数调用处获取的结果都是
None

函数

函数的返回值

注意事项

- return 关键字可以将函数定义内部的任意数据, 返回给函数调用处
- return 关键字也可以单独使用, 表示提前结束函数调用, 但不返回任何数据

```
def demo():  
    name = "admin"  
    return name  
    print(name) # 注意: return 表示返回, 后续的代码都不会被执行  
  
result = demo()  
print(result) # admin
```


案例

函数实现登录案例

需求:

- ① 当手机号为"13488888888"，返回“账号成功”；
否则返回“账号错误”。
- ② 使用assert断言实现测试结果自动比较



断言：自动判断实际结果和预期结果

用法：

① 断言相等：**assert 预期结果 == 实际结果**

② 断言包含：**assert 预期结果 in 实际结果**

结果：

通过PASS: 不显示

失败FAIL: 报错并提示 **"AssertionError"**

账户登录

 手机号/邮箱

 密码

验证码 

[忘记密码?](#)

登 录

```
def login(phone):  
    if phone == "13488888888":  
        return "账号正确"  
    else:  
        return "账号错误"  
if __name__ == '__main__':  
    assert "账号正确" == login("13488888888")  
    assert "错误" in login("")
```

07

模块和包

- 模块
- 包

学习目标

1. 掌握模块的用法
2. 掌握包的用法

模块和包

模块的概念(Module)

- 每一个以`.py`结尾的 Python 代码文件都是一个模块
- `模块名` 同样也是一个`标识符`，需要符合标识符的命名规则
- 在模块中定义的`全局变量`、`函数`、`类`都是提供给外界直接使用的`工具`
- 模块就好比是工具包, 要想使用这个工具包中的工具, 就需要先`导入模块`



导入方式

方式一: import 导入

方式二: from ... import 导入

模块和包

模块的导入方式-import

```
import 模块名1, 模块名2
```

建议：在导入模块时，每个导入独占一行

```
import 模块名1  
import 模块名2
```

如果模块的名字太长，可以使用 as 指定模块的名称，以方便在代码中的使用

```
import 模块名1 as 模块别名
```

注意：模块别名应该符合大驼峰命名法

使用方式 导入之后，通过 模块名. 使用模块提供的工具（全局变量、函数、类）

模块和包

模块的导入方式-from...import

- 如果希望从某一个模块中，导入部分工具，就可以使用 from ... import 的方式
- import 模块名 是一次性把模块中所有工具全部导入，并且通过 模块名/别名 访问

```
# 从模块导入某一个工具  
from 模块名1 import 工具名
```

使用方式 不需要通过 模块名.调用，可以直接使用模块提供的工具（全局变量、函数、类）

注意事项

1. 允许一行导入多个工具名: from 模块名 import 工具名1, 工具名2
2. 如果两个模块, 存在同名的工具, 那么后导入模块的工具, 会覆盖掉先导入的工具

模块和包

模块的导入顺序

Python 解释器在导入模块时的顺序是：

1. 查找当前目录下指定模块名的文件, 如果有就直接导入
2. 当前目录下如果没有, 则再查找系统目录下的模块



此处老司机经常翻车!

注意：给模块命名时, 务必不要和系统模块重名, 否则会直接影响系统模块的导入!

模块和包

`__name__` 属性

```
# 根据 __name__ 判断是否执行下方代码
if __name__ == "__main__":
    pass
```

作用: 处于该条件下的代码, 只有是在当前模块内执行时, 才会被运行

原理: Python 内置属性 `__name__` 的主要作用是记录一个字符串信息

- 当被其他模块导入后运行时, `__name__` 属性值为模块名
- 当在模块内运行时, `__name__` 属性值为 `__main__`

注意 一般情况下, 都需要将模块内的调试代码置于该条件下, 以防止被其他模块导入后, 调试代码也参与执行

包：(Package)是一个包含多个模块的特殊目录

- 目录下有一个特殊的文件 ``__init__.py``
- 包名 的命名方式和 变量名 一致
- 作用：python文件较多时，**方便分目录管理维护**

提示：在 PyCharm 中支持直接创建包, 工程根目录上鼠标右键 -> New -> Python Package

导包的常用方式

```
# 方式一
import 包名.模块名
包名.模块名.工具名

# 方式二
from 包名 import 模块名
模块名.工具名

# 方式三
from 包名.模块名 import 工具名
工具名
```


包操作案例

需求:

1、新建两个包：api（定义函数）和scripts（测试代码调用函数）

2、在api下，新建文件login.py，实现需求：

① 当手机号为"13488888888"，输出“账号成功”；否则“账号错误”

3、在scripts下，新建文件login_test.py，完成对登录函数的测试，测试数据为：

```
test_data = [{"username": "13488888888", "password": "123456"}, {"username": "",  
"password": "123456"}]
```

4、实现对于每一次测试结果



断言：自动判断实际结果和预期结果

用法：

① 断言相等：**assert 预期结果 == 实际结果**

② 断言包含：**assert 预期结果 in 实际结果**

结果：

通过PASS: 不显示

失败FAIL: 报错并提示 **"AssertionError"**

案例演练

包操作案例

案例演练

```
# /api/login.py
```

```
def login(phone):  
    if phone == "13488888888":  
        return "账号正确"  
    else:  
        return "账号错误"
```

```
# 导包
```

```
from api.login import login
```

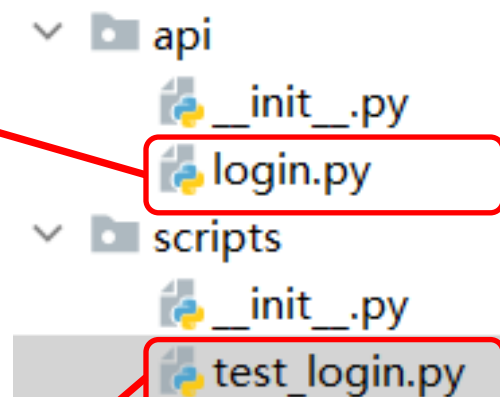
```
# 调用
```

```
print(login("13488888888"))
```

```
# 断言 assert
```

```
assert "账号正确666" == login("13488888888")
```

注意：导入方式有多种实现



 练习

• 登录案例优化

需求：

1、定义函数实现需求：当手机号为“13488888888”且密码为“123456”时，输出“登录成功”；
否则“手机号或密码错误”。

2、设计测试用例，并准备测试数据，要求：使用**列表嵌套字典**存储测试数据

示例：**[{ “username” : “13488888888” , “password” : “123456” } ,
{ “username” : “” , “password” : “123456” }]**

3、遍历测试数据并调用登录函数进行测试账号和密码校验校验

4、使用assert实现自动断言

账户登录

 手机号/邮箱

 密码

验证码



[忘记密码?](#)

登 录

 练习

• 综合练习

需求：定义函数 func，可以按照如下要求生成随机数：

1. 该函数可以接收三个参数；
 - 参数1，为生成随机数的个数；
 - 参数2 和 参数3 为生成的随机数的范围；
 - 参数2 和 参数3，如果不传递，默认生成 1- 1000 之间的随机数
2. 要求生成的随机数不能重复。
3. 将生成的随机数列表进行返回。

```
import random

def func(count, start=1, end=1000):

    data_list = []

    while True:
        num = random.randint(start, end)
        if num not in data_list:
            data_list.append(num)
            if len(data_list) == count:
                return data_list

if __name__ == '__main__':
    print(func(10, 1, 20))
```

08

变量进阶

- 引用的概念
- 可变和不可变类型
- 局部变量和全局变量

学习目标

1. 理解引用的概念
2. 掌握可变和不可变类型
3. 掌握局部变量和全局变量的用法

变量的进阶

引用的概念

提示：使用 `id()` 函数可以查看变量中保存数据所在的 **内存地址**

- **变量** 和 **数据** 是分开存储的
- **数据** 保存在内存中的一个位置
- **变量** 中保存着数据在内存中的地址
- 变量中记录数据的地址，就叫做 **引用**

注意

当给一个变量重新赋值的时候，**本质上是修改了数据的引用**

- 变量 不再 对之前的数据引用
- 变量 改为 对新赋值的数据引用

变量的引用示例

定义一个变量a，并赋值为1

a = 1



将变量a赋值为2

a = 2



将变量a的值赋值给b

b = a

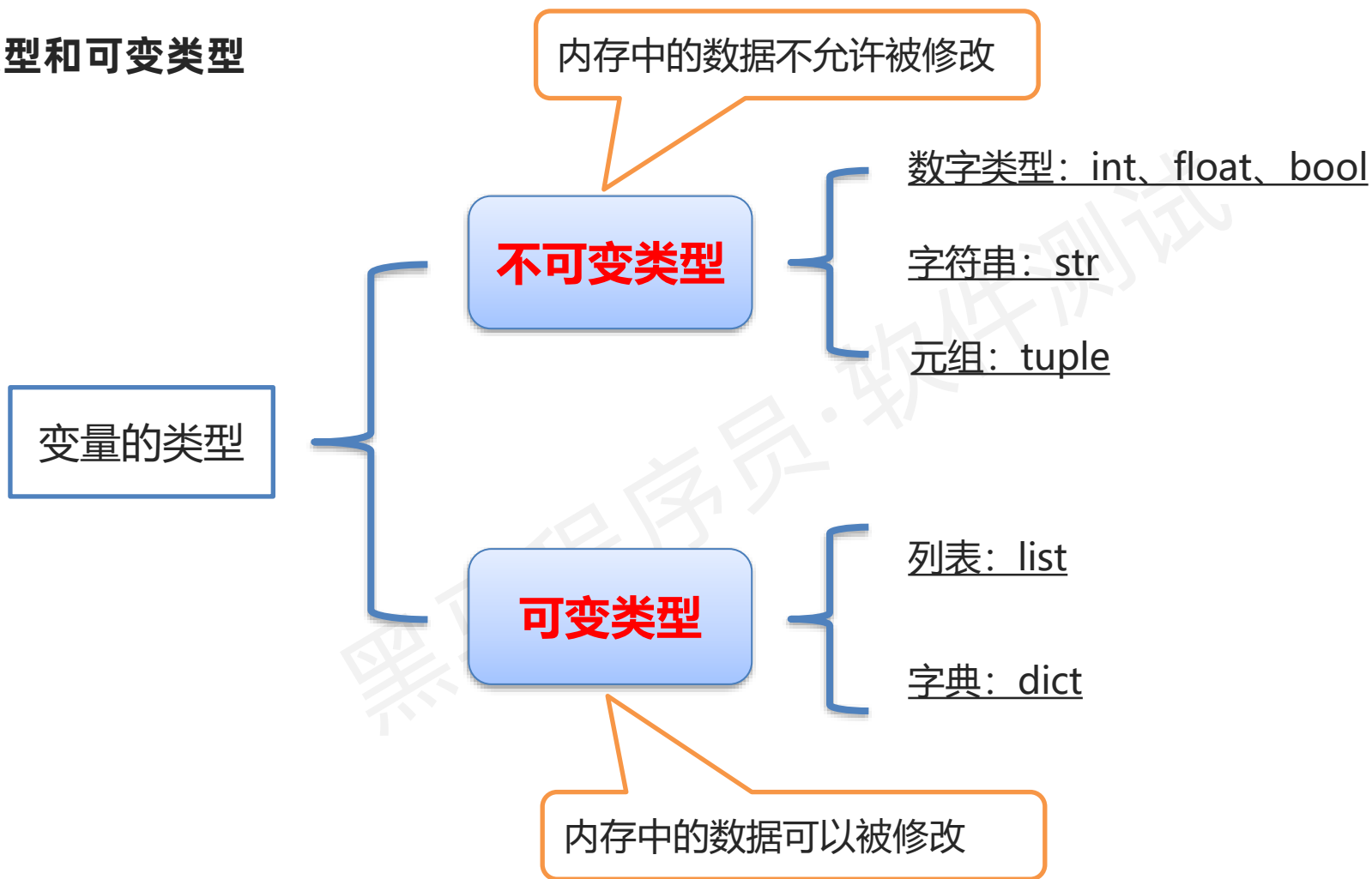


函数的参数和返回值的传递

说明 在Python中, 函数的 **参数** 和 **返回值** 传递都是引用信息

```
def test(num):  
    print(f"{num} 在函数内的内存地址是 {id(num)}")  
    result = 100  
    print(f"返回值 {result} 在内存中的地址是 {id(result)}")  
    return result  
  
a = 10  
print(f"调用函数前 内存地址是 {id(a)}")  
r = test(a)  
print(f"调用函数后 实参内存地址是 {id(a)}")  
print(f"调用函数后 返回值内存地址是 {id(r)}")
```


不可变类型和可变类型



变量的进阶

不可变类型和可变类型

不可变类型

```
a = 10
print(id(a))

a = "hello"
print(id(a))

a = "hello" + "666"
print(id(a))
```

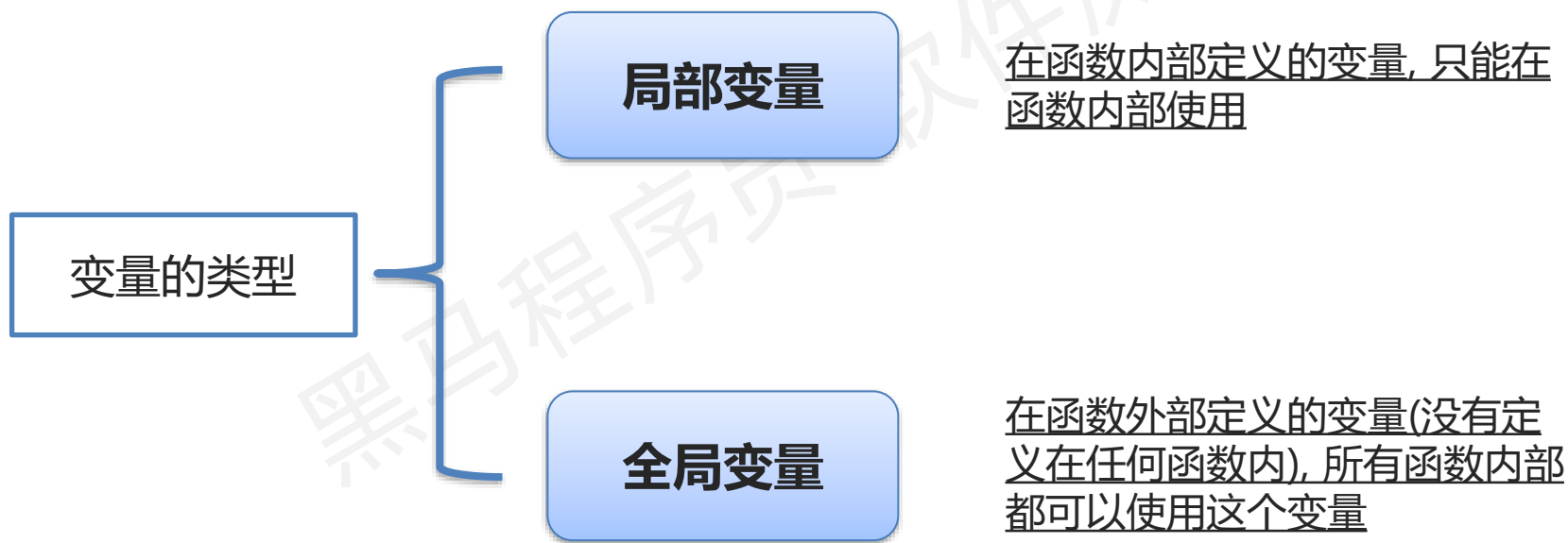
可变类型

```
demo_list = [1, 2, 3]
print(f"定义列表后的内存地址 {id(demo_list)}")
demo_list.append(999)
print(f"修改数据后的内存地址 {id(demo_list)}")

demo_dict = {"name": "小明"}
print(f"定义字典后的内存地址 {id(demo_dict)}")
demo_dict["age"] = 18
demo_dict["name"] = "老王"
print(f"修改数据后的内存地址 {id(demo_dict)}")
```

变量的进阶

局部变量和全局变量



注意 在开发中, 大多不推荐使用全局变量, 可变范围太大, 会导致程序不好维护!

变量的进阶

局部变量

- 局部变量是在函数内部定义的变量, 只能在函数内部使用
- 函数执行结束后, 局部变量会被系统回收
- 不同的函数, 可以定义相同的名字的局部变量, 彼此之间不会产生影响

生命周期

- 局部变量在函数执行时才会被创建
- 函数执行结束后 局部变量 被系统回收
- 局部变量在生命周期内, 可以用来存储 函数内部临时使用到的数据

生命周期: 就是变量从被创建到被系统回收的过程

全局变量

全局变量：是在函数外部定义的变量, 所有函数内部都可以使用

```
num = 10 # 定义一个全局变量
```

```
def demo1():  
    print("demo1.num=", num)
```

```
def demo2():  
    print("demo2.num=", num)
```

```
if __name__ == '__main__':  
    demo1()  
    demo2()
```

函数中变量获取顺序:

1. 首先查找函数内部是否存在指定名称的局部变量, 如果有直接使用
2. 内部如果没有, 则查找函数外部是否存在指定名称的全局变量, 如果有直接使用
3. 如果还没有, 则程序报错

近水楼台先得月!

在函数内部修改全局变量的值

```
num = 10

def demo1():
    # 只是定义了一个局部变量，不会修改到全局变量
    num = 100
    print("demo1.num=", num)

def demo2():
    print("demo2.num=", num)

demo1()
demo2()
```

提示：为了保证所有的函数都能够正确使用到全局变量，应该将全局变量定义在其他函数的上方

```
num = 10

def demo1():
    num = 100
    print("demo1.num=", num)

def demo2():
    global num # 告诉解释器 num 是一个全局变量
    num = 200
    print("demo2.num=", num)

demo1()
demo2()
print("num=", num)
```

注意：在函数中需要修改全局变量，需要使用 **global** 进行声明

09

函数进阶

- 缺省参数
- 多值参数
- 匿名函数

学习目标

1. 掌握缺省参数的用法
2. 知道多值参数的用法
3. 掌握如何定义匿名函数

函数进阶

缺省参数

缺省参数：定义函数时，给某个参数指定一个默认值，具有默认值的参数

说明：

- 调用函数时，如果没有传入缺省参数的值，则使用默认值
- 将常见的值设置为参数的缺省值，从而**简化函数的调用**

```
num_list = [6, 3, 9]
```

```
# 默认就是升序排序，因为这种应用需求更多
```

```
num_list.sort()
```

```
# 只有当需要降序排序时，才需要传递 `reverse` 参数
```

```
num_list.sort(reverse=True)
```


函数进阶

缺省参数

语法格式

```
def print_info(name, gender="男生"):  
    print(f"{name} 是 {gender}")
```

注意

1. 必须保证 缺省参数 在参数列表末尾
2. 在 调用函数时，如果有 多个缺省参数，需要指定参数名，这样解释器才能够知道参数的对应关系

函数进阶

缺省参数

```
def print_info(name, title="", gender="男生"):
    """
    :param title: 职位
    :param name: 班上同学的姓名
    :param gender: 性别
    """
    print(f"{title}{name} 是 {gender}")

# 提示：在指定缺省参数的默认值时，应该使用最常见的值作为默认值！
print_info("小明")
print_info("老王", title="班长")
print_info("小美", gender="女生")
```

函数进阶

多值参数

说明 如果一个函数能够处理的参数个数是不确定的, 可以考虑使用多值参数

语法格式

```
def demo(num, *args):  
    print(num)  
    print(args)  
  
demo(1, 2, 3, 4, 5)
```

提示: *args 用于接收元组类型数据, args为习惯命名,可以自定义

案例

多值参数

需求:

1. 定义一个函数 `sum_numbers`，可以接收的 任意多个整数
2. 功能要求：将传递的 所有数字累加 并且返回累加结果

```
def sum_numbers(*args):  
    num = 0  
    # 遍历 args 元组求和  
    for n in args:  
        num += n  
    return num  
  
print(sum_numbers(1, 2, 3))
```

函数进阶

元组的拆包



思考：在调用带有多值参数的函数时，如何将一个**元组变量**直接传递给args参数？



回答：可以使用 **拆包**，简化参数的传递

元组拆包的方式：在元组变量前，增加一个 *

 案例

元组拆包

需求:

1. 定义一个函数 `sum_numbers`，可以接收的 任意多个整数
2. 定义一个元组 `(1, 2, 3)`，调用上面定义好的函数对元组中所有数据进行求和

 案例

元组拆包

```
def sum_numbers(*args):  
    num = 0  
    for n in args:  
        num += n  
    return num  
  
# 需要将一个元组变量传递给函数对应的参数  
nums = (1, 2, 3)  
result = sum_numbers(*nums)  
print(f"result={result}")
```

匿名函数

lambda 参数1, 参数n : 表达式

- 用 lambda 关键字创建的简化型函数, 不同于使用 def 声明的函数, 匿名函数没有函数名称
- 匿名函数能接收任何数量的参数, 但只能返回一个表达式的值

```
sum = lambda arg1, arg2: arg1 + arg2
```

```
# 调用sum函数
```

```
print("result=", sum(10, 20)) # result= 30
```

```
print("result=", sum(20, 20)) # result= 40
```


案例

匿名函数

需求:

1. 已知用户信息数据如下:

```
user_list = [  
    {'name': '张三', 'age': 22, 'title': '测试工程师'},  
    {'name': '李四', 'age': 24, 'title': '开发工程师'},  
    {'name': '王五', 'age': 21, 'title': '测试工程师'}  
]
```

2. 要求按照年龄对用户信息数据进行排序

 案例**匿名函数**

```
user_list = [  
    {'name': '张三', 'age': 22, 'title': '测试工程师'},  
    {'name': '李四', 'age': 24, 'title': '开发工程师'},  
    {'name': '王五', 'age': 21, 'title': '测试工程师'}  
]  
  
# 按照age进行排序  
user_list.sort(key=lambda x: x["age"])  
print(user_list)
```



目录

Contents

- ◆ Python基础
- ◆ 数据序列
- ◆ 函数
- ◆ 异常



异常

- 异常的概念
- 捕获异常
- 异常的传递
- 抛出异常

学习目标

1. 理解异常的概念
2. 掌握如何捕获异常
3. 掌握异常的传递
4. 知道如何抛出异常

异常的概念

- 程序在运行时, 如果Python解释器遇到到一个错误, 则会停止程序的执行, 并且提示一些错误信息, 这就是**异常**.
- 程序停止执行并且提示错误信息这个动作, 通常称之为: 抛出 (raise) 异常



提示:

- 程序开发时, 很难将 所有的特殊情况 都处理的面面俱到, 通过 **异常捕获** 可以针对突发事件做集中的处理, 从而保证程序的 稳定性和健壮性
- 在自动化测试过程中, 也可以借助捕获异常操作, 完成类似: 用例执行报错时截图、打印日志信息等操作

捕获异常

基本语法格式

```
try:  
    # 尝试执行的代码  
except:  
    # 出现异常时执行的代码
```

说明:

- try: 尝试, 下方编写要尝试代码, 不确定是否能够正常执行的代码
- except: 如果有异常捕获, 下方编写捕获到异常, 处理失败的代码

案例

捕获异常案例

需求:

1. 获取用户从键盘输入的数据
2. 转换数据类型为整数
3. 数据转换类型正确时, 输出数据内容
4. 数据转换类型错误时, 提示输入正确数据

```
def input_num():  
    """定义输入数据函数"""  
  
    try:  
        num = int(input('请输入数字:'))  
        print('获取的数据为: {}'.format(num))  
    except:  
        print('请输入正确数据!')  
  
if __name__ == '__main__':  
    input_num()
```

捕获特定异常类型

语法格式

```
try:  
    # 尝试执行的代码  
except 异常类型:  
    # 出现异常时执行的代码
```

注意:

1. 捕获特定异常类型, 能够实现仅在特定异常类型出现时, 代码再做对应的处理
2. 具体的异常类型可以先行从代码执行的报错信息中获取, 在设置捕获异常操作

案例

捕获特定异常类型案例

需求:

1. 提示用户输入一个整数
2. 使用 8 除以用户输入的整数并且输出

案例分析:

- 用户输入的可能不是一个整数
- 0 不能做除数

```
def input_num():  
    """定义输入数据函数"""  
  
    try:  
        num = int(input("请输入整数: "))  
        result = 8 / num  
        print(result)  
    except ValueError:  
        print("请输入正确的整数")  
    except ZeroDivisionError:  
        print("除 0 错误")  
  
if __name__ == '__main__':  
    input_num()
```

捕获未知类型异常

语法格式

```
try:  
    # 尝试执行的代码  
except Exception as e:  
    # 出现异常时执行的代码  
    print('异常信息为: {}'.format(e))
```

说明:

- 如果希望程序无论出现任何错误，都不会因为Python解释器抛出异常而被终止，可以捕获 Exception
- except Exception as e: e表示捕获到的异常对象，记录异常的错误信息，e为惯用变量名，可以自定义

案例

捕获未知类型异常案例

需求:

1. 提示用户输入一个整数
2. 使用 8 除以用户输入的整数并且输出
3. 要求: 为了简化代码, 统一捕获所有异常类型

```
def input_num():  
    """定义输入数据函数"""  
  
    try:  
        num = int(input("请输入整数: "))  
        result = 8 / num  
        print(result)  
    except Exception as e:  
        print("出错了: {}".format(e))  
  
if __name__ == '__main__':  
    input_num()
```

异常捕获完整语法

```
try:
    # 尝试执行的代码
    pass
except 错误类型1:
    # 针对错误类型1, 对应的代码处理
    pass
except 错误类型2:
    # 针对错误类型2, 对应的代码处理
    pass
except Exception as e:
    # 打印错误信息
    print(e)
finally:
    # 无论是否有异常, 都会执行的代码
    pass
```

说明:

- finally: 无论是否出现异常都会执行

案例

捕获未知类型异常案例

需求:

1. 获取用户输入的数字
2. 判断获取的数字是否整数
3. 如果不是整数, 提示输入错误
4. 如果是整数, 则进一步判断是奇数还是偶数
5. 最终提示: 程序运行结束

```
def input_num():  
    """定义输入数据函数"""  
    try:  
        num = int(input('请输入数字:'))  
    except ValueError:  
        print('输入错误!')  
    else:  
        if num % 2 == 0:  
            print('获取的数据为: {}, 是偶数!'.format(num))  
        else:  
            print('获取的数据为: {}, 是奇数!'.format(num))  
    finally:  
        print('程序运行结束!')  
  
if __name__ == '__main__':  
    input_num()
```

抛出异常

应用场景：

- 在开发中，除了 代码执行出错 Python 解释器会 抛出 异常之外
- 还可以根据 应用程序 特有的业务需求 主动抛出异常

案例：提示用户 输入密码，如果 长度少于 8，抛出 异常



抛出异常

- Python 中提供了一个 Exception 异常类
- 在开发时, 如果满足特定业务需求时希望抛出异常, 可以:
 1. 创建一个 Exception类的 对象
 2. 使用 raise 关键字 抛出 异常对象

```
# 创建异常对象  
ex = Exception( "异常描述信息" )  
  
# 抛出异常对象  
raise ex
```



案例 抛出异常案例

需求:

1. 定义 input_password 函数，提示用户输入密码
2. 如果用户输入长度 < 8，抛出异常
3. 如果用户输入长度 >= 8，返回输入的密码

```
def input_password():  
    # 1. 提示用户输入密码  
    pwd = input("请输入密码: ")  
  
    # 2. 判断密码长度，如果长度 >= 8，返回用户输入的密码  
    if len(pwd) >= 8:  
        return pwd  
  
    # 3. 密码长度不够，需要抛出异常  
    # 1> 创建异常对象 - 使用异常的错误信息字符串作为参数  
    ex = Exception("密码长度不够")  
  
    # 2> 抛出异常对象  
    raise ex  
  
try:  
    user_pwd = input_password()  
    print(user_pwd)  
except Exception as result:  
    print(f"发现错误: {result}")
```




思考

1. 什么是异常?
2. 如何捕获异常?
3. 如何主动抛出一个异常?



传智教育旗下高端IT教育品牌